Phala Network 실제 코드 구현 분석

1. 개요

이 문서는 **실제 소스 코드를 직접 분석**하여 Phala Network의 구현 세부사항을 다룹니다. 앞선 문서들에서 추정했던 내용들을 실제 코드로 검증하고 보완합니다.

2. Phala Pallets 실제 구현 분석

2.1 Registry 팰릿 - 워커 등록 시스템

실제 워커 정보 구조체

```
// pallets/phala/src/registry.rs에서 실제 구현
#[derive(Encode, Decode, TypeInfo, Clone, Debug, Default)]
pub struct WorkerInfoV2<AccountId> {
pub pubkey: WorkerPublicKey,
pub ecdh_pubkey: EcdhPublicKey,
pub runtime_version: u32,
pub last_updated: u64,
pub operator: Option<AccountId>,
pub attestation_provider: Option<AttestationProvider>,
pub confidence_level: u8,
pub initial_score: Option<u32>,
```

```
pub  features: Vec<u32>,
}
```

실제 구현의 특징:

- `confidence_level` : 0-255 범위의 신뢰도 점수
- `features` : 워커가 지원하는 기능들의 벡터
- `initial_score` : 벤치마크 초기 점수 (옵션)
- `attestation_provider` : IAS, DCAP, Root 등 증명 제공자

실제 워커 등록 함수 구현

```
// register_worker_v2 함수의 실제 구현 (일부)
#[pallet::call_index(7)]
#[pallet::weight({0})]
pub  fn  register_worker_v2(
origin: OriginFor<T>,
pruntime_info: WorkerRegistrationInfoV2<T::AccountId>,
attestation: Box<Option<AttestationReport>>,
) -> DispatchResult {
ensure_signed(origin)?;
// 1. 현재 시간 가져오기
let  now = T::UnixTime::now().as_secs().saturated_into::<u64>();
// 2. 런타임 정보 해시 계산
let  runtime_info_hash = crate::hashing::blake2_256(&Encode::encode(&pruntime_in
// 3. Attestation 검증
```

```
let attestation_report = crate::attestation::validate(
*attestation,
&runtime_info_hash,
now,
T::VerifyPRuntime::get(),
PRuntimeAllowList::<T>::get(),
T::NoneAttestationEnabled::get(),
).map_err(Into::<Error<T>>::into)?;


// 4. 파라체인 ID 검증
ensure!(
pruntime_info.para_id == T::ParachainId::get(),
Error::<T>::ParachainIdMismatch
);
// 5. 제네시스 블록 해시 검증 (프로덕션에서 중요)
if T::VerifyRelaychainGenesisBlockHash::get() {
let genesis_block_hash = pruntime_info.genesis_block_hash;
let allowlist = RelaychainGenesisBlockHashAllowList::<T>::get();
ensure!(
allowlist.contains(&genesis_block_hash),
Error::<T>::GenesisBlockHashRejected
);
}
```

```
// 6. 합의 버전 추적
MaxKnownPRuntimeConsensusVersion::<T>::mutate(|info| {
use core::cmp::Ordering::*;
match  info.version.cmp(&pruntime_info.max_consensus_version) {
Less => {
info.version = pruntime_info.max_consensus_version;
info.count = 1;
}
Equal => {
info.count += 1;
}
Greater => {}
}
});


// 7. 워커 정보 업데이트 또는 생성
let  pubkey = pruntime_info.pubkey;
Workers::<T>::mutate(pubkey, |v| {
match  v {
Some(worker_info) => {
// 기존 워커 업데이트
worker_info.last_updated = now;
```

```
worker_info.operator = pruntime_info.operator;

worker_info.runtime_version = pruntime_info.version;

worker_info.confidence_level = attestation_report.confidence_level;

worker_info.features = pruntime_info.features;

// ... 이벤트 발생

}

None => {

// 새 워커 등록

*v = Some(WorkerInfoV2 {

pubkey,

ecdh_pubkey: pruntime_info.ecdh_pubkey,

runtime_version: pruntime_info.version,

last_updated: now,

operator: pruntime_info.operator,

attestation_provider: Some(attestation_report.provider),

confidence_level: attestation_report.confidence_level,

initial_score: None,

features: pruntime_info.features,

});

// ... 이벤트 발생 및 블록 번호 기록

}

}

});
```

```
Ok(())
}
```

실제 구현에서 발견한 중요한 점들:

1. **이중 검증 시스템**: IAS와 DCAP 모두 지원
2. **합의 버전 추적**: 네트워크 전체의 pRuntime 버전 분포 추적
3. **제네시스 블록 검증**: 올바른 체인에 연결되었는지 확인
4. **기능 벡터**: 워커가 지원하는 기능들을 동적으로 관리

2.2 게이트키퍼 관리 시스템

실제 게이트키퍼 등록 구현

```
// register_gatekeeper 함수의 실제 구현
#[pallet::call_index(3)]
pub fn register_gatekeeper(
origin: OriginFor<T>,
gatekeeper: WorkerPublicKey,
) -> DispatchResult {
T::GovernanceOrigin::ensure_origin(origin)?;


// 1. 마스터 키 로테이션 중에는 게이트키퍼 변경 금지
let rotating = MasterKeyRotationLock::<T>::get();
ensure!(rotating.is_none(), Error::<T>::MasterKeyInRotation);
```

```
let mut gatekeepers = Gatekeeper::<T>::get();
// 2. 첫 번째 게이트키퍼가 아니라면 마스터 키가 있어야 함
ensure!(
gatekeepers.is_empty() || GatekeeperMasterPubkey::<T>::get().is_some(),
Error::<T>::MasterKeyUninitialized
);


if !gatekeepers.contains(&gatekeeper) {
let worker_info = Workers::<T>::get(gatekeeper)
.ok_or(Error::<T>::WorkerNotFound)?;
gatekeepers.push(gatekeeper);
let gatekeeper_count = gatekeepers.len() as u32;
Gatekeeper::<T>::put(gatekeepers);


// 3. 첫 번째 게이트키퍼와 추가 게이트키퍼 구분 처리
if gatekeeper_count == 1 {
Self::push_message(GatekeeperLaunch::first_gatekeeper(
gatekeeper,
worker_info.ecdh_pubkey,
));
} else {
Self::push_message(GatekeeperChange::gatekeeper_registered(
gatekeeper,
```

```
worker_info.ecdh_pubkey,
));
}
}

Self::deposit_event(Event::<T>::GatekeeperAdded { pubkey: gatekeeper });
Ok(())
}
```

마스터 키 로테이션 구현

```
// rotate_master_key 함수의 실제 구현
#[pallet::call_index(5)]
pub fn rotate_master_key(origin: OriginFor<T>) -> DispatchResult {
T::GovernanceOrigin::ensure_origin(origin)?;

// 1. 현재 로테이션이 진행 중인지 확인
let rotating = MasterKeyRotationLock::<T>::get();
ensure!(rotating.is_none(), Error::<T>::MasterKeyInRotation);

// 2. 모든 게이트키퍼의 신원 정보 수집
let gatekeepers = Gatekeeper::<T>::get();
let gk_identities = gatekeepers
```

```
.iter()
.map(|gk| {
let  worker_info = Workers::<T>::get(gk)
.ok_or(Error::<T>::WorkerNotFound)?;
Ok(WorkerIdentity {
pubkey: worker_info.pubkey,
ecdh_pubkey: worker_info.ecdh_pubkey,
})
})
.collect::<Result<Vec<WorkerIdentity>, Error<T>>>()?;


// 3. 로테이션 ID 생성 및 잠금 설정
let  rotation_id = RotationCounter::<T>::mutate(|counter| {
*counter += 1;
*counter
});


MasterKeyRotationLock::<T>::put(Some(rotation_id));
// 4. 로테이션 메시지 전송
Self::push_message(GatekeeperLaunch::rotate_master_key(
rotation_id,
gk_identities,
));
```

```
Ok(())

}
```

3. pRuntime 실제 구현 분석

3.1 메인 함수 및 초기화

실제 명령행 인자 구조

```
// standalone/pruntime/src/main.rs의 실제 Args 구조체

#[derive(Parser, Debug, Clone)]

#[clap(about = "The Phala TEE worker app.", version, author)]

struct Args {

/// Number of CPU cores to be used for mining.

#[arg(short, long)]

cores: Option<u32>,



/// Run benchmark at startup.

#[arg(long)]

init_bench: bool,



/// Allow CORS for HTTP

#[arg(long)]

allow_cors: bool,
```

```
/// Turn on /kick API
#[arg(long)]
enable_kick_api: bool,


/// Listening IP address of HTTP
#[arg(long)]
address: Option<String>,


/// Listening port of HTTP
#[arg(long)]
port: Option<u16>,


/// Listening port of HTTP (with access control)
#[arg(long)]
public_port: Option<u16>,


/// Disable checkpoint
#[arg(long)]
disable_checkpoint: bool,


/// Checkpoint interval in seconds, default to 30 minutes
#[arg(long)]
```

```
#[arg(default_value_t = 1800)]
checkpoint_interval: u64,

/// Remove corrupted checkpoint so that pruntime can restart
#[arg(long)]
remove_corrupted_checkpoint: bool,

/// Max number of checkpoint files kept
#[arg(long)]
#[arg(default_value_t = 5)]
max_checkpoint_files: u32,

/// Handover key from another running pruntime instance
#[arg(long)]
request_handover_from: Option<String>,

/// Safe mode level (0: full, 1: no dispatch, 2: no trie proofs)
#[arg(long)]
#[arg(default_value_t = 0)]
safe_mode_level: u8,

/// Disable the RCU policy to update the Phactory state.
#[arg(long)]
```

```
no_rcu: bool,


/// The timeout of getting the attestation report. (in seconds)
#[arg(long, value_parser = parse_duration, default_value = "8s")]
ra_timeout: Duration,


/// The max retry times of getting the attestation report.
#[arg(long, default_value = "1")]
ra_max_retries: u32,


/// The timeout of a single contract query in seconds. (5-600)
#[arg(long, default_value = "10")]
query_timeout: u64,
}
```

실제 초기화 로직

```
// Args를 InitArgs로 변환하는 실제 구현
impl Args {
fn to_init_args(&self) -> InitArgs {
let sgx = pal_gramine::is_gramine();
let sealing_path;
let storage_path;
if sgx {
```

```
// Gramine에서는 보안상 하드코딩된 경로 사용
sealing_path = "/data/protected_files";
storage_path = "/data/storage_files";
} else {
// 소프트웨어 모드에서는 로컬 디렉토리 사용
sealing_path = "./data/protected_files";
storage_path = "./data/storage_files";


// 디렉토리 생성
fn  mkdir(dir: &str) {
if  let Err(err) = std::fs::create_dir_all(dir) {
panic!("Failed to create {dir}: {err:?}");
}
}
mkdir(sealing_path);
mkdir(storage_path);
}
let  cores: u32 = self.cores.unwrap_or_else(|| num_cpus::get() as  _);
InitArgs {
sealing_path: sealing_path.into(),
storage_path: storage_path.into(),
init_bench: self.init_bench,
version: env!("CARGO_PKG_VERSION").into(),
```

```
git_revision: git_revision_with_ts().to_string(),
enable_checkpoint: !self.disable_checkpoint,
checkpoint_interval: self.checkpoint_interval,
remove_corrupted_checkpoint: self.remove_corrupted_checkpoint,
max_checkpoint_files: self.max_checkpoint_files,
cores,
public_port: self.public_port,
safe_mode_level: self.safe_mode_level,
no_rcu: self.no_rcu,
ra_timeout: self.ra_timeout,
ra_max_retries: self.ra_max_retries,
query_timeout: self.query_timeout.clamp(5, 600), // 5-600초로 제한
}
}
}
```

실제 메인 함수 구현

```
#[rocket::main]
async fn main() -> Result<(), rocket::Error> {
pal_gramine::print_target_info();

let sgx = pal_gramine::is_gramine();
```

```
logger::init_subscriber(sgx);

serve(sgx).await

}


#[tracing::instrument(name = "main", skip_all)]

async fn serve(sgx: bool) -> Result<(), rocket::Error> {

let args = Args::parse();


info!(sgx, "Starting pruntime...");


// Rocket 환경 변수 설정

if let Some(address) = &args.address {

env::set_var("ROCKET_ADDRESS", address);

}

if let Some(port) = &args.port {

env::set_var("ROCKET_PORT", port.to_string());

}


let init_args = args.to_init_args();

let cores = init_args.cores;


info!("init_args: {:#?}", init_args);

// 핸드오버 요청 처리
```

```
if  let Some(from) = args.request_handover_from {
info!(%from, "Starting handover");
if pal_gramine::is_dcap() {
handover::dcap_handover_from(&from, init_args)
.await
.expect("Handover failed");
} else {
handover::handover_from(&from, init_args)
.await
.expect("Handover failed");
}
info!("Handover done");
return Ok(());
}
// Phactory 초기화
if  let Err(err) = runtime::ecall_init(init_args) {
panic!("Initialize Failed: {err:?}");
}


// 벤치마크 스레드 시작
for  i  in  0..cores {
thread::Builder::new()
.name(format!("bench-{i}"))
```

```
.spawn(move || {
set_thread_idle_policy();
// 벤치마크 루프 실행
})
.expect("Failed to spawn benchmark thread");
}
// HTTP 서버 시작
// ...
}
```

4. 실제 구현에서 발견한 핵심 특징들

4.1 보안 중심 설계

1. 다중 검증 시스템

```
// 실제 코드에서 보이는 다층 검증
- Attestation 검증 (IAS/DCAP)
- 제네시스 블록 해시 검증
- 파라체인 ID 검증
- 합의 버전 호환성 검증
- 마스터 키 로테이션 잠금
```

2. 안전 모드 지원

```
// safe_mode_level의 실제 의미
- 0: 모든 기능 활성화
- 1: 블록 동기화만, 메시지 디스패치 비활성화
- 2: 트라이 증명 저장 및 메시지 디스패치 비활성화
```

3. 체크포인트 시스템

```
// 실제 체크포인트 설정
- 기본 간격: 30분 (1800초)
- 최대 파일 수: 5개
- 손상된 체크포인트 자동 제거 옵션
```

4.2 성능 최적화

1. 스레드 관리

```
// 실제 스레드 구성
- 메인 스레드: HTTP 서버 및 제어
- 벤치마크 스레드: CPU 코어 수만큼 생성
- 각 스레드는 idle 정책 적용
```

2. 쿼리 타임아웃

```
// 실제 타임아웃 설정
```

```
query_timeout: self.query_timeout.clamp(5, 600)

// 5초 ~ 10분 사이로 제한
```

4.3 운영 편의성

1. 핸드오버 시스템

```
// 실제 핸드오버 지원

- DCAP 핸드오버: dcap_handover_from()

- 일반 핸드오버: handover_from()

- 무중단 키 이전 지원
```

2. 디버깅 지원

```
// 실제 디버깅 기능

- Git 리비전 추적

- 상세한 로깅 (tracing 크레이트)

- 버전 일관성 검증
```

5. 실제 코드와 추정 코드 비교

5.1 정확했던 추정들

- ✅ 팰릿 구조 및 기본 기능
- ✅ SGX/Gramine 지원

- ✅ HTTP API 서버 (Rocket)
- ✅ 워커 등록 및 관리 시스템

5.2 실제와 달랐던 부분들

- ❌ 워커 상태는 enum이 아닌 여러 필드로 관리
- ❌ 체크포인트는 30분 간격 (예상보다 길음)
- ❌ 안전 모드가 3단계로 세분화
- ❌ 핸드오버 시스템이 DCAP/일반으로 분리

5.3 새로 발견한 기능들

- 🆕 합의 버전 추적 시스템
- 🆕 마스터 키 로테이션 잠금 메커니즘
- 🆕 기능 벡터를 통한 동적 기능 관리
- 🆕 쿼리 타임아웃 자동 클램핑

6. 결론

실제 코드 분석을 통해 Phala Network의 구현이 **매우 정교하고 보안 중심적**으로 설계되었음을 확인할 수 있었습니다. 특히:

1. **다층 보안 검증**: 단순한 attestation 검증을 넘어 다양한 보안 검증 단계
2. **운영 안정성**: 체크포인트, 핸드오버, 안전 모드 등 운영 편의 기능
3. **성능 고려**: 스레드 관리, 타임아웃 제한 등 성능 최적화
4. **확장성**: 기능 벡터, 합의 버전 추적 등 미래 확장성 고려

이러한 실제 구현 분석을 통해 앞선 문서들의 추정 내용을 보완하고, Phala Network의 실제 동작 방식을 더 정확히 이해할 수 있게 되었습니다.